



Shiraz University

Faculty of Electrical and Computer Engineering

Microprocessors Laboratory

Project Report

Smart Safe with Remote Alert System

Dual-ATmega32 Access Control and Remote Alarm Platform

Student Name:	Shayan Niakousari
Student Number:	40236297
Course:	Microprocessors
Simulation Tool:	Proteus Design Suite

1. Introduction

This report documents a Smart Safe with Remote Alert system, implemented and simulated in Proteus using two Atmel ATmega32 microcontrollers operating as a Master and a Slave unit. The Master unit controls the physical safe: it accepts a 4-digit passcode from a matrix keypad, drives a character LCD, actuates a servo-controlled lock, logs security events to an external EEPROM, and monitors the enclosure for tampering. The Slave unit acts as a remote annunciator: it receives status messages from the Master over a UART link and reproduces them as visual (LCD, LED) and audible (buzzer) alerts at a separate location, so that a security event at the safe is also visible away from it.

The firmware for both units is written in bare-metal AVR C (no Arduino framework, no `delay.h`), targeting a 1 MHz clock to match Proteus's default ATmega32 configuration. All timing is derived from a software millisecond counter driven by Timer0, which lets the code use non-blocking, `millis()`-style timing throughout instead of long blocking delay loops.

2. System Overview

The system is split into two independent ATmega32 boards that communicate over a single asynchronous serial link (USART, 9600 baud, 8N1):

- Master unit — owns the keypad, the main LCD (LCD1), the servo lock, the tamper sensor, the external SPI EEPROM event log, the internal EEPROM passcode storage, the local LED/buzzer indicators, the sleep/wake logic and the watchdog timer.
- Slave unit — owns a second LCD (LCD2), a second LED and a second buzzer, and does nothing but listen on UART and react to the text messages the Master sends (e.g. tamper alerts, lockout notices, access events).

Communication is one-directional in practice: the Master sends short human-readable status lines terminated by `\r\n` (for example `ALERT:TAMPER DETECTED\r\n`), and the Slave parses each line with simple substring matching to decide which alert screen and which alarm output to activate.

The schematic below is the actual Proteus capture used for this project.

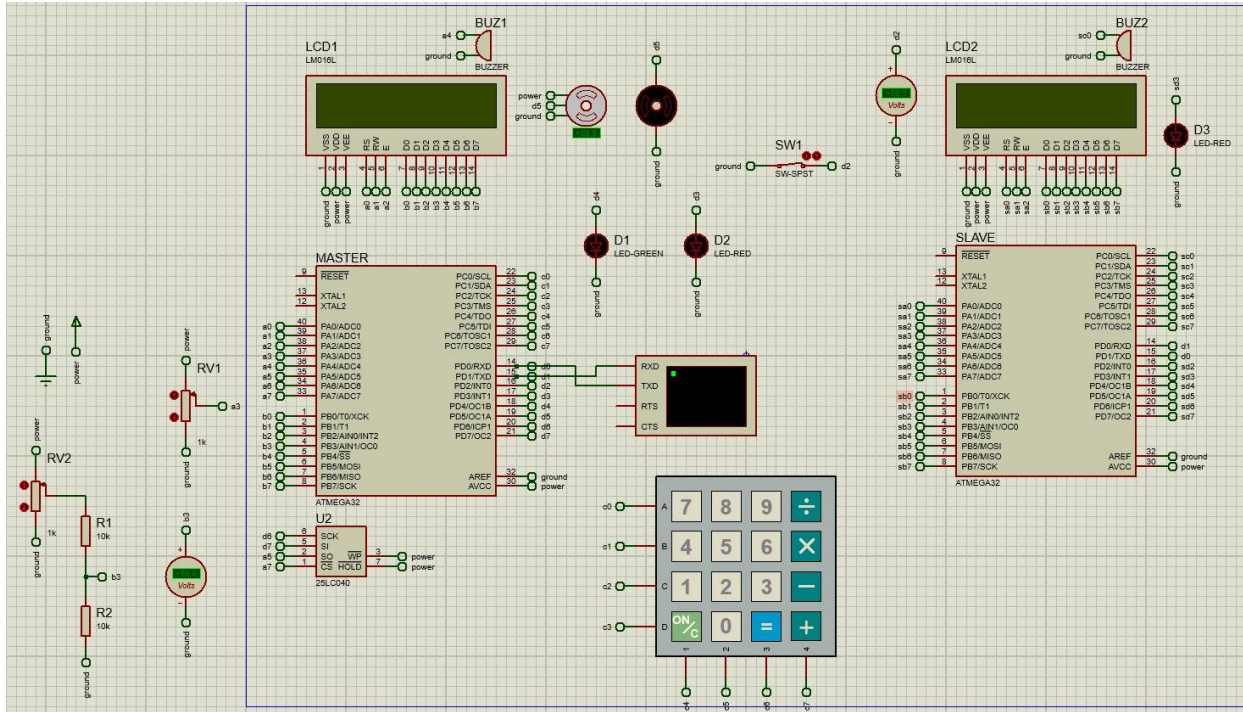


Figure 1 — Proteus schematic: Master and Slave ATmega32 boards, keypad, LCDs, EEPROM, tamper potentiometer, servo, LEDs and buzzers.

3. Hardware and Pin Mapping

Pin assignments below are taken directly from the constants defined at the top of each source file and cross-checked against the schematic in Figure 1.

3.1 Master Unit Pin Map

Peripheral	Signal	ATmega32 Pin
LCD1 (16x2)	RS / RW / E	PA0 / PA1 / PA2
LCD1 (16x2)	D0–D7 (8-bit bus)	PB0–PB7
Keypad (4x4)	Rows A–D	PC0–PC3 (output, active-low scan)
Keypad (4x4)	Columns 1–4	PC4–PC7 (input, pull-up)
EEPROM 25LC040	SCK / SI(MOSI)	PD6 / PD7
EEPROM 25LC040	SO(MISO) / CS	PA5 / PA7
Tamper sensor	Potentiometer wiper	PA3 (ADC channel 3)
LEDs	Green / Red	PD4 / PD3
Wake button	INT0	PD2
Servo (door lock)	PWM (OC1A)	PD5
Buzzer	Drive	PA4
USART	TXD / RXD	PD1 / PD0 (to Slave RXD/TXD)

3.2 Slave Unit Pin Map

Peripheral	Signal	ATmega32 Pin
LCD2 (16x2)	RS / RW / E	PA0 / PA1 / PA2
LCD2 (16x2)	D0–D7 (8-bit bus)	PB0–PB7
Remote LED	Red	PD3

Remote buzzer	Drive	PC0
USART	RXD / TXD	PD0 / PD1 (to Master TXD/RXD)

Both units run from the same 1 MHz clock assumption, which fixes the USART UBRR value at 12 for 9600 baud (double-speed mode, U2X=1), and fixes the Timer0 tick configuration used for millisecond timing (see Section 4.1).

4. Master Unit — Function-by-Function Description

The Master source file (master.c, 991 lines) is organised into 17 numbered sections. The description below follows the same order and explains every function's purpose, its inputs/outputs, and the reasoning behind its implementation.

4.1 Section 2 — SysTick (Timer0-based millis)

Because delay.h is intentionally not used, all timing in the project is built on a free-running millisecond counter driven by Timer0. This gives non-blocking, comparison-based timing (comparable to Arduino's millis()) instead of long blocking loops, which is important because the main loop must keep servicing the keypad, the watchdog and the tamper sensor while it waits.

systick_init(void)

Configures Timer0 in CTC mode with a prescaler of 8. At 1 MHz this gives a 125 kHz timer clock; with OCR0 = 124 the compare match fires every 125 counts, i.e. exactly once per millisecond. It also enables the Output Compare interrupt (OCIE0) and resets the millisecond counter to zero.

ISR(TIMER0_COMP_vect)

The Timer0 compare-match interrupt handler. It does the minimum possible amount of work — incrementing the global counter g_millis — to keep interrupt latency low.

millis(void)

Returns the current value of g_millis. Because g_millis is a 32-bit value being read outside the interrupt context, the function disables interrupts (cli()), copies the value, and restores the previous interrupt state (SREG) before returning, which prevents a torn read if the ISR fires mid-copy on an 8-bit MCU.

delay_ms(uint16_t ms)

A blocking delay implemented purely by polling millis() until the requested number of milliseconds has elapsed. It is a thin convenience wrapper kept for the handful of places (mainly LCD timing) where a short blocking wait is acceptable; everywhere else the code uses millis() directly for non-blocking waits.

4.2 Section 3 — LCD Driver (8-bit mode)

Drives a standard HD44780-compatible 16x2 character LCD (LCD1) in 8-bit parallel mode, using RS/RW/E control lines and an 8-bit data bus.

lcd_pulse_enable(void)

Generates the Enable (E) pulse the LCD controller needs to latch whatever is currently on RS/RW/data: it raises E, holds it briefly using a handful of NOP instructions for timing margin, then lowers it again.

`lcd_command(uint8_t cmd)`

Sends a control byte to the LCD (RS=0). After the pulse, it waits 2 ms for the two commands that need a longer settle time — Clear Display (0x01) and Return Home (0x02) — and 1 ms for all other commands, matching the HD44780 datasheet timing.

`lcd_data(uint8_t data)`

Sends one character byte to the LCD (RS=1) and waits 1 ms for it to be latched.

`lcd_string(const char *str)`

Writes a null-terminated C string to the LCD one character at a time by repeatedly calling `lcd_data()`.

`lcd_clear(void)`

Convenience wrapper that issues the Clear Display command (0x01).

`lcd_gotoxy(uint8_t col, uint8_t row)`

Moves the LCD cursor to a given column/row. Row 0 maps to DDRAM address 0x80 + col, row 1 maps to 0xC0 + col, which is the standard address split for a two-line HD44780 display.

`lcd_init(void)`

Performs LCD start-up: sets the control pins and the full 8-bit data bus as outputs, waits 20 ms for the LCD's internal power-on reset to finish, then issues the standard initialization sequence — 8-bit/2-line/5x7 font (0x38), display on with cursor and blink off (0x0C), auto-increment with no shift (0x06), and a final clear (0x01).

4.3 Section 4 — Keypad Driver (4x4 matrix)

Reads a 4x4 matrix keypad by driving one row low at a time and reading which column line goes low in response.

`keypad_init(void)`

Configures PC0–PC3 (rows) as outputs and PC4–PC7 (columns) as inputs, then enables the internal pull-up resistors on both the row and column pins so unpressed columns read logic-high by default.

`keypad_scan(void)`

Scans the keypad by, for each of the 4 rows in turn: setting all rows high, then pulling just that row low, waiting a few NOPs for the signal to settle, and reading the column nibble. If any column line has gone low, the corresponding row/column pair is looked up in the `keymap[][]` table and that character is returned immediately. If no key is pressed anywhere, it returns 0 after restoring all rows high. The `keymap` itself lays out digits 0–9 alongside 'C', '=', and arithmetic symbols, of which the code only actually uses the digits and 'C' (clear).

4.4 Section 5 — USART Driver (interrupt-driven RX)

Implements a 9600-baud, 8N1 USART link with an interrupt-driven receive ring buffer, used both to send status/log messages to the Slave unit and to accept a simple one-character command ('L') for dumping the event log.

usart_init(void)

Sets the baud-rate registers (UBRR = 12, matching double-speed mode at 1 MHz for 9600 baud), enables the double-speed bit (U2X), enables the receiver, transmitter and receive-complete interrupt, and configures the frame format as 8 data bits, no parity, 1 stop bit.

ISR(USART_RXC_vect)

Fires whenever a byte arrives. It reads the byte from UDR and pushes it into a 32-byte circular buffer (rx_buf), advancing the write index (rx_head). If the buffer is full (the next write index would collide with the read index, rx_tail) the incoming byte is silently dropped rather than overwriting unread data.

usart_send_char(char c)

Blocks until the transmit data register is empty (UDRE flag) and then writes the byte, i.e. a standard busy-wait single-byte transmit.

usart_send_string(const char *str)

Transmits a null-terminated string by calling usart_send_char() for each character.

usart_available(void)

Returns true if the ring buffer's read and write indices differ, i.e. there is at least one unread byte.

usart_read_char(void)

Pops and returns the next byte from the ring buffer, advancing the read index; returns 0 if the buffer is empty.

4.5 Section 6 — Software SPI, External EEPROM (25LC040) and Event Log

The hardware SPI pins (PB5/PB6/PB7) are already used by the LCD's 8-bit data bus, so communication with the external 25LC040 SPI EEPROM is done with a bit-banged (software) SPI implementation on separate general-purpose pins. This EEPROM is used exclusively as a small circular event log (who logged in, when a tamper or lockout occurred, etc.), independent of the passcode, which is instead stored in the ATmega32's own internal EEPROM.

ee_cs_low(void) / ee_cs_high(void)

Small inline helpers that drive the EEPROM's chip-select line low (select) or high (deselect).

ee_spi_transfer(uint8_t out)

Performs one full-duplex SPI byte transfer in software: for each of the 8 bits, most-significant-bit first, it sets the MOSI (SI) line to the corresponding bit of out, pulses SCK high, samples the MISO (SO) line into the

return value, then pulses SCK low again, with short NOP delays for signal settling. This single routine underlies every EEPROM command, address and data byte that is sent or received.

ee_spi_init(void)

Configures the SCK and SI (MOSI) pins as outputs (SO/MISO as input), and configures and raises the CS pin so the EEPROM starts deselected.

ee_write_enable(void)

Sends the WREN (Write Enable) opcode, which the 25LC040 requires before any write can succeed — write operations are otherwise ignored by the chip.

ee_read_status(void)

Sends the RDSR (Read Status Register) opcode and returns the status byte, whose bit 0 (WIP — write in progress) indicates whether an internal write cycle is still running.

ee_wait_ready(void)

Polls ee_read_status() in a loop until the WIP bit clears, i.e. blocks until the previous write cycle (up to ~5 ms on this part) has completed.

ee_addr_msb_bit(uint16_t addr)

The 25LC040 only has an 8-bit address byte in its opcode stream, so the 9th address bit (needed to address the full 512-byte part) is folded into the opcode itself. This helper returns 0x08 if bit 8 of the address is set, or 0 otherwise, ready to be OR'd into the READ/WRITE opcode.

ee_write_byte(uint16_t addr, uint8_t data)

Performs a full single-byte write: sends WREN, selects the chip, sends the WRITE opcode (with the address MSB folded in), the low address byte, then the data byte, deselected the chip, and finally waits for the write cycle to finish via ee_wait_ready().

ee_read_byte(uint16_t addr)

Performs a full single-byte read: selects the chip, sends the READ opcode (with address MSB folded in) and the low address byte, clocks out one dummy byte (0xFF) to receive the data byte, then deselected the chip and returns the value read.

log_init(void)

Reads the stored log-record count from a fixed EEPROM address (EE_LOG_COUNT_ADDR). If the EEPROM is blank (reads as 0xFF, its erased state), the count is treated as zero and re-initialised, which handles the case of a brand-new or blank-simulated EEPROM.

log_event(event_code_t code, uint32_t timestamp, uint8_t extra)

Appends one 6-byte record (1 byte event code, 4 bytes big-endian timestamp, 1 byte extra data) to the circular log area starting at EE_LOG_START. If the log is full (log_count has reached

EE_LOG_MAX_RECORDS) it wraps back to index 0, overwriting the oldest record — a simple ring-buffer log. After writing the record it increments and persists the record count.

event_name(uint8_t code)

Translates a numeric event_code_t value into a short human-readable string (e.g. "LOGIN_OK", "TAMPER", "LOCKOUT") for use when the log is dumped over USART.

log_dump_via_usart(void)

Walks every stored log record, reads back its code/timestamp/extra fields from the external EEPROM, formats each one into a single CSV-style line with sprintf(), and transmits it over USART, bracketed by START/END markers. This is triggered remotely by sending the character 'L' to the Master (see handle_usart_commands() in Section 4.11).

4.6 Section 7 — ADC (Tamper Potentiometer)

A potentiometer standing in for a physical tamper/vibration sensor is read on ADC channel 3 (PA3).

adc_init(void)

Selects AVCC as the analog reference and sets the ADC prescaler to 64, which brings the ADC clock to 125 kHz — inside the 50–200 kHz range the ATmega32 requires for full conversion accuracy at this main clock — and enables the ADC.

adc_read(uint8_t channel)

Selects the requested ADC input channel, starts a single conversion, busy-waits on the ADSC flag until the conversion finishes, and returns the 10-bit result.

4.7 Section 8 — Power-Fail Detection (Analog Comparator)

This subsystem uses the ATmega32's built-in analog comparator, with AIN0 internally replaced by the 1.22 V bandgap reference, to detect a sagging supply voltage on a resistor-divider node.

power_fail_init(void)

Configures PB3 (AIN1) as a floating input and sets ACBG (Analog Comparator Bandgap Select), so the comparator's non-inverting input becomes the internal 1.22 V reference instead of the physical AIN0 pin.

power_fail_detected(void)

Returns true if the comparator output (ACO) is set, meaning the bandgap voltage now exceeds the voltage on the divider node, i.e. the monitored supply has sagged below the threshold set by the divider.

Note: in the current build, power_fail_init() is never called from main(), and its polling counterpart check_power_fail() (Section 4.14) is likewise not invoked from the main loop — both are left in place but disabled, with a comment in the code explaining that PB3 is already used as the LCD's D3 data-bus line in this particular wiring, so the analog comparator input would conflict with the LCD. The functions remain fully implemented and only need to be re-enabled if the pin conflict is resolved (e.g. by moving the LCD to a 4-bit bus).

4.8 Section 9 — Servo PWM (Timer1, Door Lock)

The door lock is a hobby servo driven by a 50 Hz PWM signal on OC1A (PD5), generated with Timer1 in Fast PWM mode.

`servo_init(void)`

Configures PD5 as an output and sets up Timer1 in Fast PWM mode with TOP defined by ICR1 (SERVO_TOP = 2499). With a prescaler of 8 at 1 MHz, the timer counts at 125 kHz, so a TOP of 2499 gives a PWM frame period of 2500 counts / 125 kHz = 20 ms, i.e. the standard 50 Hz servo refresh rate. OC1A is configured for non-inverting output, and the pulse width is initialised to the locked position.

`servo_lock(void) / servo_unlock(void)`

Set OCR1A to SERVO_PULSE_LOCK (125 counts $\approx$ 1.0 ms) or SERVO_PULSE_UNLOCK (250 counts $\approx$ 2.0 ms) respectively, moving the servo horn to the locked or unlocked angle.

4.9 Section 10 — Watchdog and Sleep Mode

This section handles two related low-level concerns: recovering safely if the firmware ever hangs (via the hardware watchdog), and putting the MCU into a low-power state when the safe has been idle, waking it again on a button press.

`power_mgmt_capture_reset_source(void)`

Called at the very start of main(), before anything else. It checks the MCU Control and Status Register (MCUCSR) for the Watchdog Reset Flag (WDRF); if set, it records that the last reset was caused by the watchdog (g_was_wdt_reset) so this can be logged later, then clears MCUCSR so stale reset-cause flags don't linger into the next reset.

`watchdog_init(void)`

Enables the hardware watchdog timer with a roughly 2-second timeout (WDTO_2S). If the main loop ever stops calling watchdog_kick() — e.g. because of a firmware lock-up — the watchdog will reset the MCU automatically.

`watchdog_kick(void)`

Resets (feeds) the watchdog timer, called frequently throughout the main loop and inside every long polling loop (lockout countdown, tamper alarm, door-open wait) so those intentional multi-second waits don't themselves trigger a watchdog reset.

`ISR(INT0_vect)`

The external interrupt handler for the wake button. Its body is intentionally empty — the only job of this interrupt is to bring the MCU out of power-down sleep; no further action is needed inside the ISR itself.

`wake_button_init(void)`

Configures the wake button pin as an input with its pull-up enabled, clears any pending INTO flag, and enables the INTO interrupt in low-level-sensitive mode — the only trigger mode by which the ATmega32 can wake from full power-down sleep via an external interrupt.

`enter_sleep_mode(void)`

Puts the MCU into Power-down sleep to conserve energy while idle. It disables the watchdog first (so it can't fire while asleep), selects `SLEEP_MODE_PWR_DOWN`, and then performs the sequence recommended by the datasheet to avoid a race condition: interrupts are disabled, any pending INTO flag is cleared, sleep is armed (`sleep_enable()`), interrupts are re-enabled, and the SLEEP instruction is executed immediately afterwards — this ordering guarantees no interrupt can slip in between arming sleep and actually sleeping. After waking, it disables sleep, disables INTO again (so a held-down button doesn't keep re-triggering the low-level interrupt), waits until the wake button is physically released, and finally re-enables the watchdog.

4.10 Section 11 — LEDs / Buzzer Helpers

`leds_init(void)`

Configures the green and red status LED pins as outputs and ensures both start off.

`buzzer_init(void)`

Configures the buzzer pin as an output and ensures it starts off.

`buzzer_on(void) / buzzer_off(void)`

Simple bit-set/bit-clear helpers that turn the local buzzer on or off.

4.11 Section 12 — Passcode / Internal EEPROM

The 4-digit passcode and the count of consecutive failed attempts are stored in the ATmega32's internal EEPROM (via `avr/eeprom.h`), which persists across power cycles, separate from the external SPI EEPROM used purely for the event log.

`reset_entry(void)`

Clears the in-progress digit-entry buffer (`entered[]`, `entered_len`), clears the LCD, and redraws the "Enter code:" prompt with the cursor positioned on the second line ready for masked digit entry.

`check_passcode(const char *digits)`

Reads the 4 stored passcode digits out of internal EEPROM and compares them one-by-one against the 4 ASCII digits just entered on the keypad. As a defensive measure against a blank/corrupt EEPROM, any stored digit greater than 9 is treated as if it were (`index+1`) rather than trusted as-is, before the comparison. Returns 1 only if all four digits match, 0 otherwise.

`passcode_eeprom_init(void)`

Called once at startup to make sure the stored passcode is sane. It reads the four stored digits and checks whether every one is a valid decimal digit (0–9); if any is not (which happens, for example, when Proteus

loads only the compiled .hex and leaves EEPROM content as the erased value 0xFF), it overwrites the stored passcode with the default 1-2-3-4. It performs the same kind of sanity check and reset-to-zero for the stored failed-attempt counter.

get_fail_count(void)

Reads the failed-attempt counter from internal EEPROM, treating an invalid value (0xFF, or anything above the configured maximum) as zero.

set_fail_count(uint8_t v)

Persists a new failed-attempt count to internal EEPROM.

4.12 Section 13 — Alarm / Access Handlers

These are the higher-level state-transition functions triggered by the outcome of a passcode entry or a tamper detection. Each one drives the local LEDs/buzzer/LCD, sends a status message to the Slave over USART, and writes an event record to the external EEPROM log.

trigger_tamper_alarm(void)

Turns on the red LED and local buzzer, sends "ALERT:TAMPER DETECTED" over USART, logs an EVT_TAMPER record with the current uptime in seconds, and shows a "!! TAMPER !!" message on the LCD. It then holds this alarm state for a fixed duration (TAMPER_ALARM_MS = 5 s), calling watchdog_kick() throughout so the wait doesn't trip the watchdog, before switching the buzzer and LED back off.

trigger_lockout(void)

Called after too many wrong passcode attempts. It turns on the red LED and buzzer, sends "ALERT:LOCKOUT MAX ATTEMPTS" over USART, and logs an EVT_LOCKOUT record. It then runs a real 30-second countdown timed off millis() (not a blocking delay), redrawing the LCD only when the displayed number of remaining seconds actually changes, from 30 down to 0, kicking the watchdog on every iteration; keypad input is not read during this loop, so it is effectively ignored during lockout. Once the countdown reaches zero it holds the "00" display briefly, then turns off the alarm outputs, resets the failed-attempt counter to zero, clears any partially entered digits, sends an "INFO:LOCKOUT FINISHED" message so the Slave can clear its own alarm, and redraws the normal entry prompt.

grant_access(void)

Called when the correct passcode has been entered. It turns on the green LED, sends "INFO:ACCESS GRANTED", logs an EVT_LOGIN_OK record, shows "Access Granted" on the LCD, and unlocks the servo. It then holds the door in the unlocked state for a fixed duration (DOOR_OPEN_MS = 5 s), kicking the watchdog throughout, before re-locking the servo, turning the green LED back off, and resetting the failed-attempt counter to zero.

deny_access(void)

Called when an incorrect 4-digit code has been entered. It increments and persists the failed-attempt counter, sends "INFO:ACCESS DENIED", logs an EVT_LOGIN_FAIL record (including the new fail count as

the "extra" byte), and briefly shows "Wrong Code" on the LCD. If the failed-attempt count has now reached the configured maximum (`MAX_FAILED_TRIES = 3`), it calls `trigger_lockout()`.

4.13 Section 14 — Tamper Polling

`check_tamper(void)`

Reads the current tamper-sensor ADC value. On its very first call it simply records that reading as a baseline and reports no tamper event (there is nothing yet to compare against). On every later call, it computes the absolute difference between the new reading and the previous one; if that difference exceeds `TAMPER_THRESHOLD`, it reports a tamper event (returns 1). Either way, the new reading becomes the baseline for the next comparison, so the check is really watching for sudden changes in the sensor value rather than an absolute level, which naturally rejects a fixed/steady analog input as "no tamper".

4.14 Section 15 — Power-Fail Monitoring

`check_power_fail(void)`

Reads the analog-comparator power-fail flag and reacts only on state transitions: the first time a failure is seen after being previously normal, it sends "ALERT:POWER FAIL", logs an `EVT_POWER_FAIL` record, and lights the red LED; the first time normal power is seen again after a failure, it sends "INFO:POWER RESTORED", logs an `EVT_POWER_RESTORED` record, and turns the red LED back off. As noted in Section 4.7, this function exists and is fully implemented, but is not currently called from `main()` because its underlying pin (PB3) is shared with the LCD data bus in this wiring.

4.15 Section 16 — USART Command Handling

`handle_usart_commands(void)`

Checks whether a byte has arrived on USART and, if it has, reads it; if that byte is the letter 'L' (upper or lower case), it calls `log_dump_via_usart()` to print the entire stored event log back over the serial link. This gives a simple remote way to retrieve the log without needing a display attached to the Master.

4.16 Section 17 — `main()`

`main()` performs, in order: capturing the watchdog reset-cause flag, starting the millisecond systick and enabling global interrupts (deliberately done early because `lcd_init()` already depends on `delay_ms()`, which depends on `millis()`), then initialising the LEDs, buzzer, wake button, LCD, keypad, USART, software-SPI EEPROM interface, the event log, the ADC, the servo, and the passcode/fail-count sanity checks in internal EEPROM, and finally arming the watchdog.

After sending a couple of banner lines over USART (and a warning line plus a logged `EVT_WATCHDOG_RESET` event if the previous reset was caused by the watchdog), it draws the initial "Enter code" prompt and enters the infinite main loop, which on every iteration:

- Kicks the watchdog and processes any pending USART command (the 'L' log-dump request).

- Every TAMPER_POLL_MS (100 ms), re-checks the tamper sensor; on a positive detection it triggers the tamper alarm and resets keypad entry.
- Scans the keypad; on a keypress it waits for the key to be released (debounce), then either clears the current entry ('C'), or appends a digit. Once 4 digits have been entered, it checks the passcode and calls grant_access() or deny_access() accordingly.
- If no keypad or tamper activity has occurred for IDLE_SLEEP_MS (10 s), it sends a "SYSTEM SLEEP" notice, turns the LCD backlight/display off, re-arms the wake interrupt, and calls enter_sleep_mode(); on waking it re-enables the display, shows an "Awake" message, sends "SYSTEM AWAKE", and resets the entry prompt and idle timer.

Note that check_power_fail() is deliberately not called inside this loop, for the pin-conflict reason explained in Section 4.14.

5. Slave Unit — Function-by-Function Description

The Slave source file (slave.c, 353 lines) is considerably smaller than the Master, since its only job is to receive text over USART and turn recognised messages into local alerts. It reuses the same systick/LCD/USART driver pattern as the Master.

5.1 Section 2 — Systick

systick_init(), ISR(TIMERO0_COMP_vect), millis() and delay_ms() are implemented identically to the Master's versions described in Section 4.1, providing the same non-blocking millisecond timing base.

5.2 Section 3 — LCD Driver

lcd_pulse_enable(), lcd_command(), lcd_data(), lcd_string(), lcd_clear(), lcd_gotoxy() and lcd_init() are implemented identically to the Master's LCD driver described in Section 4.2, driving the Slave's own LCD2 display.

5.3 Section 4 — USART Driver

usart_init(), ISR(USART_RXC_vect), usart_available() and usart_read_char() are implemented identically to the Master's receive-side USART code described in Section 4.4. The Slave never needs to transmit, so it has no usart_send_char()/usart_send_string() equivalent.

5.4 Section 5 — LED / Buzzer Helpers

led_init(void)

Configures the Slave's red LED pin as an output, initially off.

buzzer_init(void)

Configures the Slave's buzzer pin as an output, initially off.

alarm_on(void) / alarm_off(void)

Turn the Slave's red LED and buzzer on or off together as a single combined "alarm output", used for every alert condition.

5.5 Section 6 — Message Handling

`contains(const char *haystack, const char *needle)`

A thin wrapper around the standard `strstr()`, returning true if needle appears anywhere inside haystack. This is how the Slave classifies an incoming line without needing a full parser.

`show_message_on_lcd(const char *msg)`

A generic fallback display routine: clears the LCD and writes up to the first 16 characters of msg on line 1 and, if the message is longer, the next 16 characters on line 2 — matching the 16x2 geometry of the LM016L display used in the schematic. This is used for any incoming message that doesn't match one of the specifically recognised alert phrases.

5.6 Section 7 — main()

`main()` initialises `systick`, enables interrupts, initialises the LED, buzzer, LCD and USART, and shows a static "Remote Alarm / Waiting..." idle screen. It then enters an infinite loop with two responsibilities running side by side:

- Message assembly and classification — every available byte is read from the USART ring buffer and appended to a local line buffer (`msg_buf`) until a carriage-return or newline is seen, at which point the accumulated line is null-terminated and classified with a sequence of `contains()` checks: "LOCKOUT MAX ATTEMPTS" shows a lockout alert and turns the alarm on; "LOCKOUT FINISHED" clears the alarm and restores the idle screen; "TAMPER" shows a tamper alert and turns the alarm on; "POWER FAIL", "FAILURE" or "BREACH" fall back to the generic `show_message_on_lcd()` display and turn the alarm on; "POWER RESTORED" clears the alarm and shows a "system normal" message; anything else not matched by the above is simply shown verbatim via `show_message_on_lcd()`.
- Automatic alarm timeout — independently of message reception, if the alarm is currently active and `ALARM_DURATION_MS` (5 s) has elapsed since it was triggered, the alarm is turned off automatically, so a missed or garbled "finished"/"restored" message from the Master can't leave the Slave's buzzer sounding indefinitely.

6. Master-Slave Communication Protocol

The two units communicate over a single UART link at 9600 baud, 8N1, with the Master's TXD wired to the Slave's RXD and vice versa (visible in Figure 1 via the shared virtual terminal probe used for debugging in the schematic). The protocol is a simple, one-directional, human-readable text protocol: the Master transmits short ASCII lines, each terminated with `\r\n`, and the Slave classifies each line by substring matching rather than a fixed binary format. This keeps both ends easy to debug directly from a serial terminal, at the cost of being less compact than a binary protocol.

The messages sent by the Master, and the Slave's reaction to each, are summarised below.

Message (from Master)	Sent when	Slave reaction
SYSTEM READY / PASSCODE: 4 DIGITS	Boot	No match — shown verbatim via fallback display
INFO:ACCESS GRANTED	Correct passcode entered	No specific match — shown verbatim via fallback display
INFO:ACCESS DENIED	Wrong passcode entered	No specific match — shown verbatim via fallback display
ALERT:LOCKOUT MAX ATTEMPTS	3rd consecutive wrong passcode	Lockout alert screen; alarm ON
INFO:LOCKOUT FINISHED	30 s lockout countdown ends	Alarm OFF; idle screen restored
ALERT:TAMPER DETECTED	Tamper sensor delta exceeds threshold	Tamper alert screen; alarm ON
ALERT:POWER FAIL	Power-fail comparator trips (currently disabled)	Fallback display; alarm ON
INFO:POWER RESTORED	Power-fail comparator clears (currently disabled)	"System normal" screen; alarm OFF
WARN:RESET BY WATCHDOG	Master rebooted due to a watchdog reset	No specific match — shown verbatim via fallback display
SYSTEM SLEEP / SYSTEM AWAKE	Master enters/exits power-down sleep	No specific match — shown verbatim via fallback display

7. Results and Observations

In Proteus simulation, the Master boots, initialises its passcode to the default 1-2-3-4 the first time it runs (since a fresh EEPROM image reads as 0xFF), and displays the "Enter code:" prompt. Entering the correct 4-digit code unlocks the servo, lights the green LED and holds the door open for 5 seconds before automatically re-locking; entering an incorrect code shows "Wrong Code" and increments a persisted fail counter, and three consecutive wrong entries trigger a real-time 30-second lockout countdown on the LCD. Nudging the tamper potentiometer produces an immediate "!!! TAMPER !!!" alert with buzzer and red LED for 5 seconds. Every one of these events is both sent as a status line to the Slave over USART and appended as a timestamped record in the external SPI EEPROM log, retrievable at any time by sending the character 'L' to the Master.

On the Slave side, the corresponding LCD alert screen and buzzer/LED activation were confirmed to follow each Master-side event, with the alarm clearing itself automatically 5 seconds after the last alert if no explicit "finished"/"restored" message arrives first.

7.1 Simulation Screenshots

The screenshots below were captured directly from the Proteus simulation and show the system passing through its main operating states, confirming that the Master, the Slave, and the virtual USART terminal all agree on the state of the system at each point.

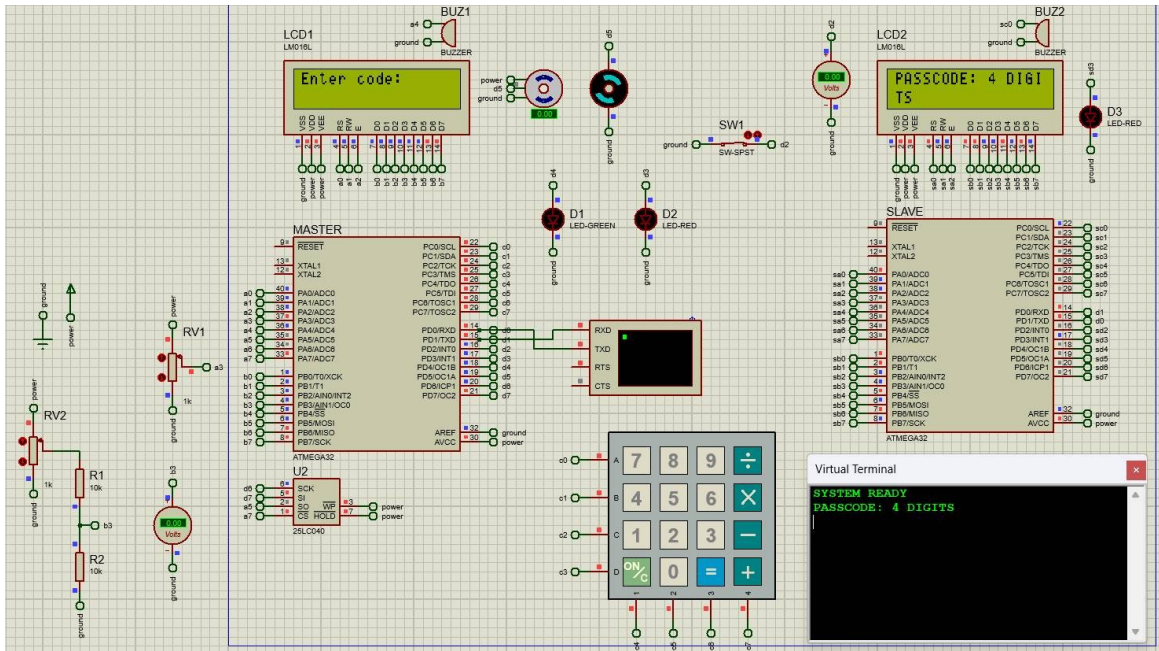


Figure 2 — Startup: the Master shows the "Enter code:" prompt, the Slave shows "PASSCODE: 4 DIGITS", and the virtual terminal confirms SYSTEM READY.

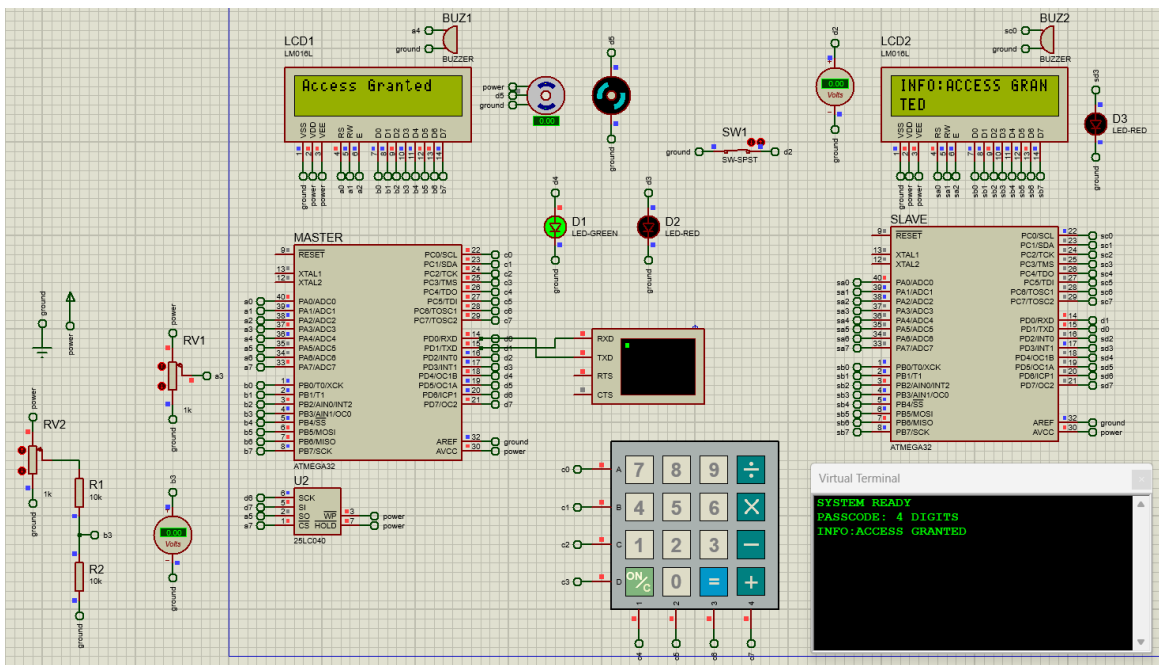


Figure 3 — Correct passcode entered: the Master displays "Access Granted" (green LED on, servo unlocked), the Slave mirrors "INFO:ACCESS GRANTED", and the terminal logs the same event.

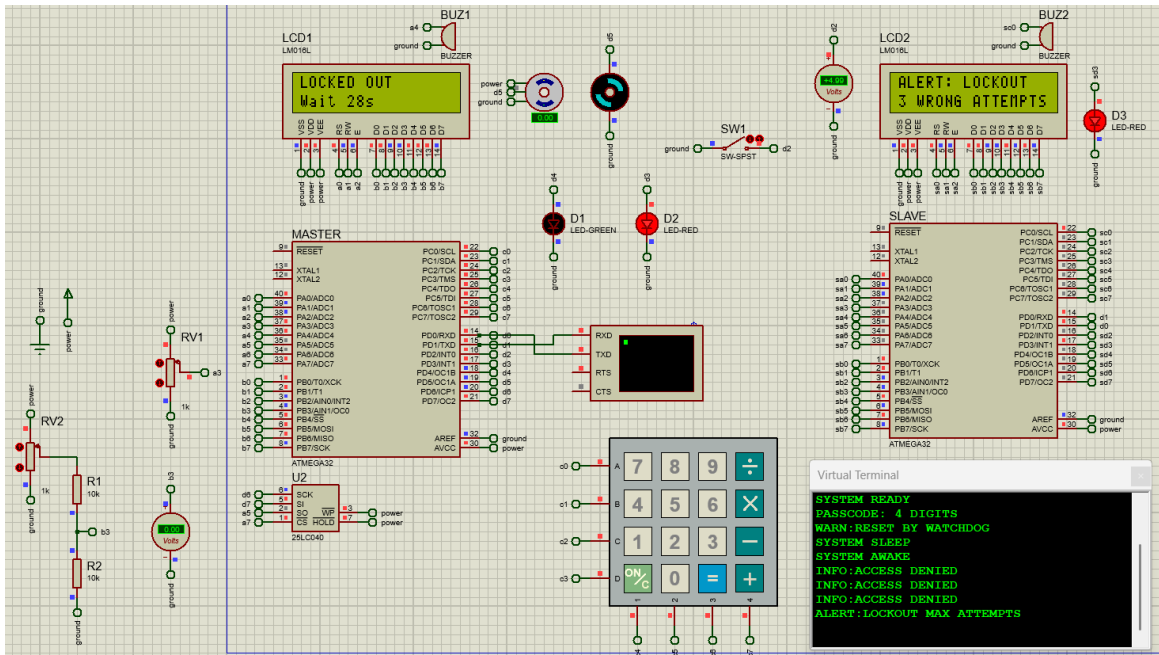


Figure 4 — Lockout after three wrong attempts: the Master shows the live "LOCKED OUT / Wait 28s" countdown while the Slave shows "ALERT: LOCKOUT / 3 WRONG ATTEMPTS"; the terminal history shows the three ACCESS DENIED lines followed by the LOCKOUT MAX ATTEMPTS alert.

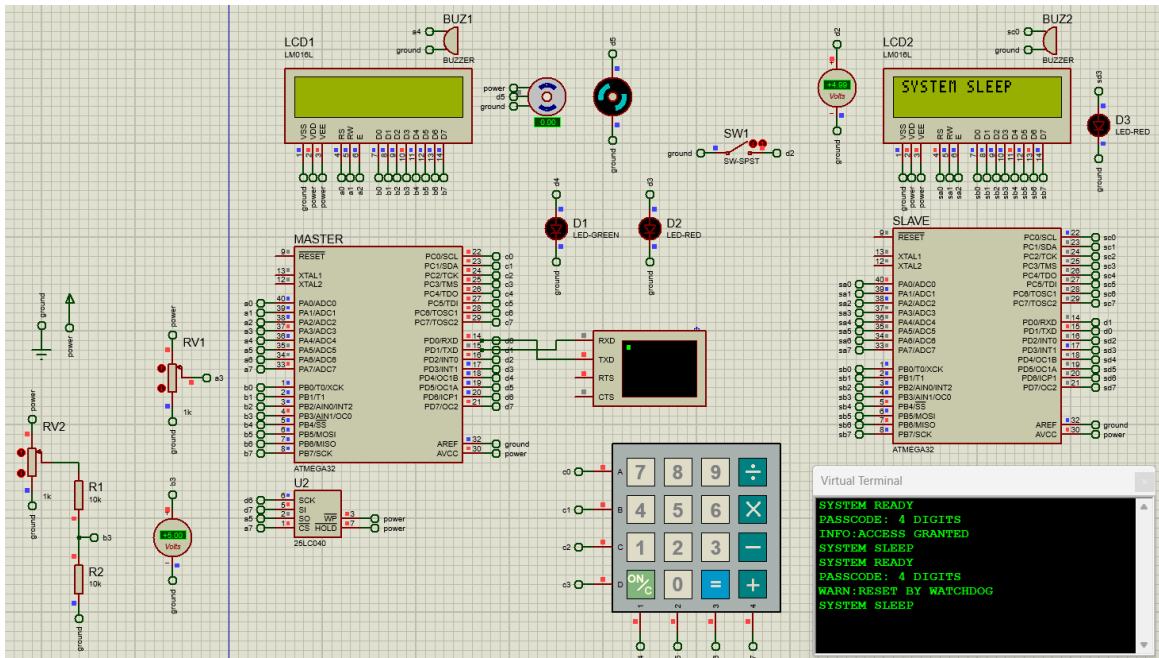


Figure 5 — Idle sleep: after a period of inactivity the Master blanks its LCD and enters power-down sleep ("SYSTEM SLEEP"), while the Slave LCD independently continues to show the last alert state.

8. Known Limitations

- Power-fail monitoring (power_fail_init() / check_power_fail()) is implemented but intentionally disabled, because its pin, PB3, doubles as the D3 line of the LCD's 8-bit data bus in this wiring —

enabling both simultaneously would conflict. Re-enabling it would require either moving the LCD to a 4-bit interface or relocating the power-fail divider to a free pin.

- The Master-to-Slave protocol is one-directional and unacknowledged: the Slave has no way to confirm receipt back to the Master, so a corrupted or dropped line is only ever recovered by the Slave's own 5-second alarm auto-timeout rather than a retransmission.
- The event log on the external EEPROM is a fixed-size ring buffer; once full, new events silently overwrite the oldest ones rather than being archived elsewhere.

9. Conclusion

The project implements a complete two-microcontroller access-control and remote-alerting system on bare-metal AVR C, covering keypad input, LCD output, EEPROM-backed persistent storage (both internal, for the passcode, and external SPI, for the event log), PWM-driven servo actuation, analog tamper sensing, UART inter-board communication, watchdog-protected operation, and low-power sleep with external-interrupt wake-up. All timing is handled through a Timer0-based software millisecond clock rather than blocking `delay.h` calls, which keeps the main loop responsive to the keypad and tamper sensor even while multi-second operations such as the lockout countdown or door-open hold are in progress.